

# Actividad 4 — Optimización PostgreSQL para Ecommify

Informe de optimización de consultas, índices especializados y particionamiento declarativo sobre la **arquitectura híbrida** del proyecto. Todas las métricas son **reales**, medidas con EXPLAIN (ANALYZE, BUFFERS) sobre PostgreSQL 16 + PostGIS y **1.000.000 de órdenes** sintéticas.

## 1. Metodología

| Aspecto       | Detalle                                                                                                                                                                  |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Motor         | PostgreSQL 16 + PostGIS 3.4 (imagen multi-arch imresamu/postgis, <b>nativa arm64</b> en Docker)                                                                          |
| Configuración | <b>Parámetros por defecto</b> de PostgreSQL (deliberado: hace visible el impacto de cada optimización)                                                                   |
| Datos         | <b>1.000.000 de órdenes</b> sintéticas (~1,49 M ítems, ~1,12 M pagos, ~610 K reseñas) generadas muestreando los CSV reales para preservar distribuciones — ver README.md |
| Esquema base  | Tabla orders <b>plana</b> y <b>sin</b> índices de optimización ni particionamiento, para aislar el "antes" (sql/00_schema_baseline.sql)                                  |
| Medición      | EXPLAIN (ANALYZE, BUFFERS); <b>se reporta Execution Time</b> y <b>bloques</b> (shared/temp)                                                                              |

**Notas de rigor.** (1) Las mediciones son de una corrida sobre el contenedor **nativo arm64** (sin emulación); los deltas observados son de **órdenes de magnitud**, muy por encima del ruido de caché. (2) La configuración por defecto y el dataset son reproducibles (--seed 42), de modo que los planes pueden regenerarse. (3) Como el dataset cabe en caché, varias consultas "antes" leen de shared buffers; aun así los Seq Scan sobre 1 M filas siguen dominando frente a los accesos por índice.

### Por qué datos sintéticos a escala

Las semillas reales tienen ~200 filas: con ese volumen el planificador descarta índices y particiones (un Seq Scan de 200 filas siempre gana), y el criterio de particionamiento (>100.000 filas) es inalcanzable. Inflar a 1 M de órdenes conservando distribuciones realistas es lo que hace que las decisiones del optimizador —y por tanto las métricas— sean significativas.

## 2. Fase 1 — Optimización de consultas por análisis de planes

### 2.1 Identificación de consultas críticas (OLTP)

El catálogo de consultas críticas (sql/01\_critical\_queries.sql) son operaciones **OLTP** de alta frecuencia de la tienda: detalle y seguimiento de pedido, historial de la cuenta, panel del vendedor, navegación de catálogo, promociones activas y los workers de fulfillment/outbox. Los planes base (results/01\_baseline\_plans.txt) revelan **dos naturalezas de cuello de botella distintas**:

| Naturaleza                                                 | Consultas                                           | Se resuelve con...                     |
|------------------------------------------------------------|-----------------------------------------------------|----------------------------------------|
| <b>Camino de acceso</b> (Seq Scan por falta de índice)     | Q2, Q4, Q5, Q7, Q8                                  | Índices especializados → <b>Fase 2</b> |
| <b>Estructura del SQL</b> (subconsulta/función/paginación) | OPT-1..4 (abajo)                                    | <b>Reescritura</b> → esta fase         |
| Ya óptimas                                                 | Q1, Q3 (PK por order_id), Q6 (GiST de la EXCLUDE) — |                                        |

Hallazgo relevante: Q1/Q3 (búsqueda por order\_id) **ya usan la PK** porque order\_id es la **columna líder** de la clave compuesta (order\_id, order\_purchase\_timestamp). El problema de "escanear todas las particiones" sin el timestamp aparece sólo al particionar (Fase 3), no en la tabla plana.

#### Descripción de las consultas críticas (sql/01\_critical\_queries.sql)

**Q1 — Detalle de orden por order\_id** · página de pedido, email de confirmación, soporte. La app suele conocer sólo order\_id.

```
SELECT * FROM orders WHERE order_id = $1;
```

**Q2 — Historial de pedidos de una persona** · página "mis pedidos", paginada y por recencia. La persona es customer\_unique\_id; las órdenes referencian customer\_id, por lo que requiere join customers → orders.

```
SELECT o.order_id, o.order_status, o.order_purchase_timestamp
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE c.customer_unique_id = $1
ORDER BY o.order_purchase_timestamp DESC
LIMIT 20;
```

**Q3 — Seguimiento de pedido por order\_id** · "rastrea tu pedido", altísima frecuencia.

```
SELECT order_status, order_delivered_customer_date, order_estimated_delivery_date
FROM orders WHERE order_id = $1;
```

**Q4 — Ítems recientes de un vendedor** · panel del vendedor, paginado.

```
SELECT oi.order_id, oi.product_id, oi.price, o.order_status, o.order_purchase_timestamp
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
                AND o.order_purchase_timestamp = oi.order_purchase_timestamp
WHERE oi.seller_id = $1
ORDER BY o.order_purchase_timestamp DESC
LIMIT 50;
```

**Q5 — Navegación de catálogo por categoría** · listado de productos, keyset.

```
SELECT product_id, product_category_name, product_weight_g
FROM products WHERE category_id = $1
ORDER BY product_id LIMIT 24;
```

**Q6 — Promociones activas de un producto** · se evalúa en cada vista de producto;  
respaldada por el índice GiST de la restricción EXCLUDE.

```
SELECT promotion_id, discount_percentage
FROM product_promotions
WHERE product_id = $1 AND promotion_period @> now();
```

**Q7 — Pedidos por aprobar** · worker de fulfillment; predicado muy selectivo (~0,5 %).

```
SELECT order_id, order_purchase_timestamp
FROM orders WHERE order_status = 'created'
ORDER BY order_purchase_timestamp LIMIT 100;
```

**Q8 — Sondeo del despachador de outbox** · patrón *transactional outbox* (~2 % pendiente).

```
SELECT event_id, payload FROM outbox_events
WHERE processed_at IS NULL ORDER BY created_at LIMIT 100;
```

Esta fase aborda las consultas cuyo cuello es la **estructura del SQL**: la mejora se obtiene reescribiendo, sin agregar índices (medido sobre el esquema base). Se aplican **4 técnicas** (sql/02\_query\_optimizations.sql, salida en results/02\_optimizations.txt).

## 2.2 Resultados (resumen)

| #     | Técnica                                                    | Antes     | Después  | Δ Tiempo        | Δ Bloques leídos                              |
|-------|------------------------------------------------------------|-----------|----------|-----------------|-----------------------------------------------|
| OPT-1 | Descorrelación: subconsulta correlacionada → JOIN+GROUP BY | 9.365 ms  | 247 ms   | -97,4 % (×37,9) | 6.595.232 → 32.944 (-99,5 %)                  |
| OPT-2 | Sargabilidad: eliminar función en WHERE                    | 59,7 ms   | 22,7 ms  | -62,0 % (×2,6)  | ~24.200 (sin cambio; ganancia de CPU)         |
| OPT-3 | Anti-join: NOT IN → NOT EXISTS                             | 21.628 ms | 79,3 ms  | -99,6 % (×273)  | 3.401.506 + 1.826.039 temp → 36.886 (-98,9 %) |
| OPT-4 | Paginación: OFFSET profundo → keyset                       | 30,1 ms   | 0,036 ms | -99,9 % (×836)  | 1.215 → 27 (-97,8 %)                          |

## 2.3 Análisis por caso

### OPT-1 — Descorrelación de subconsultas escalares

**Caso:** Reporte "ventas por vendedor" para una página de 100 vendedores.  
**Por qué era apropiado:** La forma naïf (estilo ORM) ejecuta **dos subconsultas correlacionadas por fila**; como order\_items no tiene índice por seller\_id, el plan repite un Seq Scan de 1,5 M filas **100×2 = 200 veces** (loops=100 en cada SubPlan, *Rows Removed by Filter: 1.488.856*). Es un patrón O(N×M).  
**Reescritura:** Un único Hash Right Join + GROUP BY **recorre order\_items una sola vez**.

```
-- ANTES: dos subconsultas escalares correlacionadas por vendedor
SELECT s.seller_id,
       (SELECT count(*)           FROM order_items i WHERE i.seller_id = s.seller_id) AS items_sold,
       (SELECT coalesce(sum(i.price),0) FROM order_items i WHERE i.seller_id = s.seller_id) AS revenue
FROM s100 s;
-- DESPUÉS: un solo recorrido con JOIN + GROUP BY
SELECT s.seller_id, count(i.*) AS items_sold, coalesce(sum(i.price),0) AS revenue
FROM s100 s LEFT JOIN order_items i ON i.seller_id = s.seller_id
GROUP BY s.seller_id;
```

**Resultado:** 9.365 → 247 ms (-97,4 %, ×37,9); bloques leídos 6,60 M → 33 K (-99,5 %). Cambio de plan: CTE Scan + SubPlan×2 (Seq Scan) → HashAggregate + Hash Right Join (Seq Scan único). Desaparece incluso el JIT que el coste disparaba.

### OPT-2 — Sargabilidad (eliminar función sobre la columna)

**Caso:** Conteo de estados de un día concreto.  
**Por qué era apropiado:** date\_trunc('day', order\_purchase\_timestamp) = ... evalúa la función en cada una de las 10<sup>6</sup> filas y, sobre todo, vuelve el predicado **no-sargable**: imposibilita usar índice o poda de particiones. La forma equivalente con **rango semiabierto** (>= '2018-05-10' AND < '2018-05-11') es **sargable**.

```
-- ANTES (no sargable): función sobre la columna
SELECT order_status, count(*) FROM orders
WHERE date_trunc('day', order_purchase_timestamp) = TIMESTAMPTZ '2018-05-10'
GROUP BY order_status;
-- DESPUÉS (sargable): rango semiabierto
SELECT order_status, count(*) FROM orders
WHERE order_purchase_timestamp >= TIMESTAMPTZ '2018-05-10'
AND order_purchase_timestamp < TIMESTAMPTZ '2018-05-11'
GROUP BY order_status;
```

**Resultado:** 59,7 → 22,7 ms (−62,0 %). Los bloques son idénticos (~24.200; ambos hacen Seq Scan sobre la tabla sin índice); la ganancia aquí es CPU (sin date\_trunc por fila). El valor estratégico es que **habilita** el índice (Fase 2) y la **poda de particiones** (Fase 3), donde el mismo predicado pasa a leer una fracción de la tabla.

#### OPT-3 — Anti-join (NOT IN → NOT EXISTS)

**Caso:** Job de CSAT: pedidos entregados de un día sin reseña.

**Por qué era apropiado:** NOT IN (subconsulta) no puede transformarse en un anti-join hash limpio por la semántica con NULL; el plan degenera en un SubPlan con Materialize de order\_reviews **re-evaluado** (loops=1270), con **derrame a disco** (temp read=1.826.039). NOT EXISTS permite un Parallel Hash Right Anti Join.

```
-- ANTES
SELECT count(*) FROM orders o
WHERE o.order_status = 'delivered' AND o.order_purchase_timestamp >= '2018-05-10'
AND o.order_purchase_timestamp < '2018-05-11'
AND o.order_id NOT IN (SELECT order_id FROM order_reviews);
-- DESPUÉS
SELECT count(*) FROM orders o
WHERE o.order_status = 'delivered' AND o.order_purchase_timestamp >= '2018-05-10'
AND o.order_purchase_timestamp < '2018-05-11'
AND NOT EXISTS (SELECT 1 FROM order_reviews r WHERE r.order_id = o.order_id);
```

**Resultado:** 21.628 → 79,3 ms (−99,6 %, ×273); de 3,40 M bloques compartidos + 1,83 M temporales a 37 K bloques. Además NOT EXISTS es **semánticamente correcto** ante NULL (evita el resultado vacío sorpresa de NOT IN).

#### OPT-4 — Paginación keyset (vs OFFSET profundo)

**Caso:** Navegar a una página profunda del catálogo.

**Por qué era apropiado:** OFFSET 20000 LIMIT 24 **lee y descarta 20.024 filas**

(Seq Scan + top-N heap sort) para devolver 24. El método **keyset** (WHERE product\_id > :cursor ORDER BY product\_id LIMIT 24) **arranca en el cursor** usando el índice de PK, sin descartar nada. No requiere índices nuevos.

```
-- ANTES: OFFSET profundo (lee y descarta 20.024 filas)
SELECT product_id, product_category_name FROM products
ORDER BY product_id OFFSET 20000 LIMIT 24;
-- DESPUÉS: keyset (arranca en el cursor de la página anterior)
SELECT product_id, product_category_name FROM products
WHERE product_id > :cursor ORDER BY product_id LIMIT 24;
```

**Resultado:** 30,1 → 0,036 ms (−99,9 %, ×836); 1.215 → 27 bloques. Cambio de plan: Seq Scan + Sort → Index Scan using products\_pkey. (El coste de OFFSET **crece** con la profundidad de la página; keyset es constante.)

### 2.4 Conclusión de la Fase 1

Las 4 reescrituras —sin tocar el esquema— eliminan dos patrones O(N×M) (OPT-1, OPT-3) y dos antipatrones de acceso (OPT-2, OPT-4), con reducciones de **62 % a 99,9 %** en tiempo y hasta **99,5 %** en bloques leídos. OPT-2 sienta además la base sargable que aprovechan las Fases 2 y 3.

## 3. Fase 2 — Índices especializados

Se implementan **5 tipos** de índice (el enunciado pide ≥3), cada uno elegido por el patrón que optimiza (sql/03\_specialized\_indexes.sql, **salida** en results/03\_indexes.txt y results/03b\_gin\_orders.txt). Todos atacan las consultas cuyo cuello es el **camino de acceso** (los Seq Scan detectados en la Fase 1).

Sentencias de creación:

```
-- IDX-1 B-tree simple (sirve el filtro de Q2 y el join por FK)
CREATE INDEX idx_customers_unique_id ON customers(customer_unique_id);
CREATE INDEX idx_orders_customer ON orders(customer_id);
-- IDX-2 B-tree compuesto (igualdad + orden de Q4, evita el Sort)
CREATE INDEX idx_order_items_seller_ts ON order_items(seller_id, order_purchase_timestamp DESC);
-- IDX-3 B-tree compuesto (filtro + orden de Q5, keyset)
CREATE INDEX idx_products_category_pid ON products(category_id, product_id);
-- IDX-4 Parcial (sólo el ~0,5% de Q7)
CREATE INDEX idx_orders_created_pending ON orders(order_purchase_timestamp)
WHERE order_status = 'created';
-- IDX-5 Parcial (sólo el ~2% pendiente de Q8)
CREATE INDEX idx_outbox_unprocessed ON outbox_events(created_at)
WHERE processed_at IS NULL;
-- IDX-6 GIN para contención JSONB (@>)
CREATE INDEX idx_products_specs_gin ON products USING gin (product_specifications jsonb_path_ops);
-- IDX-7 BRIN para rango temporal sobre tabla append-only (correlación física)
CREATE INDEX idx_demo_brin ON orders_ts_demo USING brin (order_purchase_timestamp) WITH (pages_per_range = 32);
```

### 3.1 Resultados (resumen)

| #     | Tipo             | Consulta                      | Antes                | Después                         | $\Delta$<br>Tiempo | Tamaño índice              | Cambio de plan                                 |
|-------|------------------|-------------------------------|----------------------|---------------------------------|--------------------|----------------------------|------------------------------------------------|
| IDX-1 | B-tree simple    | (x2) Q2 historial cliente     | 121,1 ms             | 0,31 ms                         | -99,7 %            | 34 MB + 56 MB              | 2x Seq Scan → 2x Index Scan                    |
| IDX-2 | B-tree compuesto | Q4 ítems del vendedor         | 111,5 ms             | 0,44 ms                         | -99,6 %            | 96 MB                      | Seq Scan + Sort → Index Scan (sin Sort)        |
| IDX-3 | B-tree compuesto | Q5 catálogo                   | 7,96 ms              | 0,05 ms                         | -99,4 %            | 2,1 MB                     | Index Scan(pkey)+Filter → Index Scan compuesto |
| IDX-4 | Parcial          | Q7 por aprobar                | 17,2 ms              | 0,49 ms                         | -97,2 %            | 136 kB                     | Seq Scan + top-N → Index Scan parcial          |
| IDX-5 | Parcial          | Q8 outbox                     | 16,9 ms              | 0,10 ms                         | -99,4 %            | 40 kB                      | Seq Scan → Index Scan parcial                  |
| IDX-6 | GIN              | JSONB @> selectivo (products) | 3,71 ms              | 0,56 ms                         | -85,0 %            | 176 kB                     | Seq Scan → Bitmap Index Scan                   |
| IDX-7 | BRIN vs B-tree   | rango temporal (count)        | 18,0 ms (sin índice) | 3,3 ms (BRIN) / 9,3 ms (B-tree) | —                  | BRIN 32 kB vs B-tree 21 MB | Seq Scan → BRIN Bitmap                         |

### 3.2 Documentación por índice

#### IDX-1 · B-tree simple — customers(customer\_unique\_id) + orders(customer\_id)

- **Justificación:** Q2 filtra por customer\_unique\_id (no es la PK) y une por customer\_id (columna FK que PostgreSQL **no indexa automáticamente**). Sin índices ambos lados son Seq Scan paralelos.
- **Patrón que optimiza:** Igualdad selectiva + join 1-a-muchos.
- **Trade-offs:** Indexar dos CHAR(32) sobre 1 M de filas cuesta **90 MB** y encarece cada INSERT/UPDATE de orders/customers. Se justifica por ser una consulta de altísima frecuencia (página de cuenta).
- **Impacto:** 121,1 → 0,31 ms (**x392**); Parallel Seq Scan → Index Scan en ambos.

#### IDX-2 · B-tree compuesto — order\_items(seller\_id, order\_purchase\_timestamp DESC)

- **Justificación:** Q4 hace WHERE seller\_id = ? ORDER BY order\_purchase\_timestamp DESC LIMIT 50. Con la igualdad primero y el orden después, el índice satisface filtro y orden, eliminando el Sort. El DESC evita además invertir el escaneo.
- **Trade-offs:** 96 MB (1,5 M filas). Un índice sólo por seller\_id sería menor pero dejaría el Sort; el compuesto es el equilibrio correcto para este patrón.
- **Impacto:** 111,5 → 0,44 ms (**x255**); desaparece Sort + Seq Scan.

#### IDX-3 · B-tree compuesto — products(category\_id, product\_id)

- **Justificación:** Q5 (WHERE category\_id = ? ORDER BY product\_id) se servía por la PK con Filter, descartando filas. El compuesto agrupa por categoría y deja product\_id ya ordenado (ideal para keyset).
- **Trade-offs:** Sólo 2,1 MB (tabla de 32 K); coste de escritura despreciable.
- **Impacto:** 7,96 → 0,05 ms (**x162**).

#### IDX-4 · Parcial — orders(order\_purchase\_timestamp) WHERE order\_status='created'

- **Justificación:** Q7 sólo consulta el ~0,5 % de pedidos created. Un índice parcial indexa exclusivamente esas filas → minúsculo y siempre selectivo; además ordena por timestamp (sirve el ORDER BY).
- **Trade-offs:** 136 kB frente a ~21 MB de un B-tree total sobre order\_status (~150x menos espacio y mantenimiento), a cambio de servir **sólo** ese predicado.
- **Impacto:** 17,2 → 0,49 ms (**x35**); Parallel Seq Scan → Index Scan parcial.

#### IDX-5 · Parcial — outbox\_events(created\_at) WHERE processed\_at IS NULL

- **Justificación:** El despachador (Q8) sólo lee los ~2 % de eventos pendientes. Es el índice del diseño híbrido: el conjunto "caliente" se mantiene diminuto aunque la tabla crezca sin límite.
- **Trade-offs:** 40 kB. Al procesarse un evento (processed\_at deja de ser NULL) la fila sale del índice → el índice no crece con el histórico.

- **Impacto:** 16,9 → 0,10 ms (×176).

#### IDX-6 · GIN (jsonb\_path\_ops) — products (product\_specifications)

- **Justificación:** El predicado de **contención** `product_specifications @> '{...}'` no es indexable por B-tree; GIN sí. `jsonb_path_ops` es más compacto y rápido que el GIN por defecto cuando sólo se usa `@>`.
- **Patrón que optimiza:** Filtros por atributos semiestructurados (garantía, fragilidad).
- **Impacto:** Predicado **selectivo** (~2 % = 637 filas): 3,71 → 0,56 ms (×6,6); Seq Scan → Bitmap Index Scan. Índice de 176 kB.
- **Trade-off de SELECTIVIDAD (clave):** La ventaja de GIN depende de cuántas filas empareja. Se midió también sobre `orders.metadata @> '{"channel":"app_ios","gift":true}'` (~1,3 % de 1 M = 12.790 filas, `results/03b_gin_orders.txt`): a esa selectividad la diferencia es **pequeña y depende del estado de caché** —con la tabla cacheada el Seq Scan secuencial puede ganar, y con la tabla en disco el Bitmap Index Scan evita leerla entera y gana—, porque los accesos **dispersos** al heap compiten con un barrido secuencial. Conclusión: la ventaja **clara y repetible** de GIN aparece con predicados muy selectivos (caso `products`) y/o tablas que no caben en caché; su valor **cualitativo** es que habilita consultas de contención JSONB imposibles para B-tree. Además su escritura es más costosa que la de un B-tree.

#### IDX-7 · BRIN vs B-tree — rango temporal sobre orders

- **Justificación:** Para barridos por rango de fecha sobre una tabla **append-only** (los pedidos llegan en orden temporal → alta correlación físico-lógica), BRIN guarda sólo min/máx por bloque-rango: índice **diminuto**.
- **Trade-offs medidos: BRIN 32 kB vs B-tree 21 MB (~655× menor)** y, en este caso, **más rápido** (3,3 vs 9,3 ms: BRIN lee sólo los bloque-rango relevantes). **Condición crítica:** BRIN sólo sirve si el orden físico correlaciona con la columna; por eso se midió sobre una copia ordenada que emula el orden natural de `orders` (el generador sintético barajó los timestamps). Sin correlación, BRIN degenera y escanea casi toda la tabla.
- **Impacto:** Seq Scan (18,0 ms) → BRIN Bitmap (3,3 ms), pagando **655× menos espacio** que un B-tree.

### 3.3 Conclusión de la Fase 2

Los B-tree (simple/compuesto) y parciales llevan las consultas OLTP de acceso a **sub-milisegundo** (×35 a ×392) convirtiendo Seq Scans en Index Scans; los parciales lo hacen con índices de **decenas de kB**. GIN habilita consultas JSONB imposibles para B-tree (con la salvedad de selectividad documentada en IDX-6), y BRIN ofrece indexación de rango temporal a **1/655 del tamaño** de un B-tree cuando hay correlación física. El costo transversal —mayor espacio y escrituras más lentas— se justifica por la frecuencia de cada patrón.

## 4. Fase 3 — Particionamiento declarativo

### 4.1 Análisis y selección

| Decisión             | Elección                              | Justificación                                                                                                                                 |
|----------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Tabla candidata      | <code>orders</code> (1.000.000 filas) | Supera el criterio de >100.000 filas y es la tabla de hechos central                                                                          |
| Columna de partición | <code>order_purchase_timestamp</code> | Domina los WHERE de la carga (Q7, OPT-2, OPT-3, barridos por rango, BRIN) y crece monótonamente (append-only)                                 |
| Tipo                 | <b>RANGE</b>                          | El dato es temporal y se consulta por <b>rangos</b> de fecha (descarta LIST —no son valores discretos— y HASH —no se busca reparto uniforme—) |
| Granularidad         | <b>Mensual</b>                        | ~38 K filas/partición: equilibra número de particiones y tamaño; alinea con los reportes mensuales                                            |
| Red de seguridad     | Partición <b>DEFAULT</b>              | Captura datos fuera de rango (las 1.000 filas anómalas de 2019)                                                                               |

### 4.2 Implementación y validación

`sql/04_partitioning.sql` crea `orders_part` PARTITION BY RANGE (`order_purchase_timestamp`), genera 26 particiones mensuales (2016-09 ... 2018-10) mediante un bucle DO, añade la partición DEFAULT, y migra el millón de filas con INSERT ... SELECT. Para aislar el efecto del particionamiento, **ninguna** de las dos tablas (plana vs particionada) tiene índice sobre `order_purchase_timestamp`: la mejora proviene sólo de la poda.

```
-- Tabla particionada (la clave de partición forma parte de la PK)
CREATE TABLE orders_part ( ... , PRIMARY KEY (order_id, order_purchase_timestamp) )
PARTITION BY RANGE (order_purchase_timestamp);
-- Particiones mensuales generadas en bucle:
CREATE TABLE orders_part_2017_06 PARTITION OF orders_part
  FOR VALUES FROM ('2017-06-01') TO ('2017-07-01'); -- (×26 meses)
CREATE TABLE orders_part_default PARTITION OF orders_part DEFAULT; -- red de seguridad
-- Migración:
INSERT INTO orders_part SELECT ... FROM orders;
```

**Validación estructural** (`results/04_partitioning.txt`): 26 particiones de ~38 K filas (~8 MB c/u) y la DEFAULT con **exactamente 1.000 filas** (las anómalas de 2019), confirmando que la red de seguridad funciona.

### 4.3 Comparación de rendimiento

Barrido por rango de un mes (`order_purchase_timestamp` en junio-2017), misma consulta contra ambas tablas:

```
SELECT count(*) FROM <orders | orders_part>
WHERE order_purchase_timestamp >= TIMESTAMPTZ '2017-06-01'
AND order_purchase_timestamp < TIMESTAMPTZ '2017-07-01';
```

| Escenario                         | Plan                                                                        | Tiempo Bloques leídos |         |
|-----------------------------------|-----------------------------------------------------------------------------|-----------------------|---------|
| A) Tabla plana orders             | Parallel Seq Scan de toda la tabla (Rows Removed by Filter: 320.373)        | 33,9 ms               | 26.752  |
| B) Tabla particionada orders_part | Seq Scan de una sola partición orders_part_2017_06 (25 particiones podadas) | 9,8 ms                | 1.039   |
|                                   |                                                                             | -71,0 % (×3,4)        | -96,1 % |

La poda de particiones (*partition pruning*) ocurre en tiempo de planificación: el plan de B sólo incluye la partición de junio-2017; las otras 25 ni se abren.

#### 4.4 Trade-offs (documentados honestamente)

- **Consulta sin la clave de partición** (caso C: búsqueda por `order_id`). El plan es un Append sobre las **27 particiones**, cada una con un Index Scan de su PK. Sigue siendo rápido a esta escala (0,27 ms) pero **toca todas las particiones** y el coste de **planificación** crece con su número. Mitigación: incluir la clave de partición en las consultas, o un índice global por `order_id` si el patrón es frecuente. Esto explica por qué en la tabla plana Q1/Q3 eran óptimas con la sola PK.
- **Mantenimiento**. Más objetos (27 tablas + índices) que administrar; a cambio, `VACUUM/DROP` por partición son baratos (eliminar un mes histórico es un `DROP TABLE` instantáneo en vez de un `DELETE` masivo).

#### 4.5 Estrategia de creación automática

Documentada en el script (§ final): función `ensure_next_orders_partition()` agendada con **pg\_cron** (crea el mes siguiente antes de que llegue), o **pg\_partman** con retención automática. La partición `DEFAULT` se monitorea para que permanezca casi vacía (su llenado indicaría que falta provisionar particiones).

#### 4.6 Conclusión de la Fase 3

El particionamiento `RANGE` mensual reduce el barrido por rango **-71,0 % en tiempo** y **-96,1 % en bloques** vía poda, y convierte el purgado de histórico en un `DROP` instantáneo, a cambio de mayor número de objetos y de penalizar las consultas que no incluyen la clave de partición. (La reducción de bloques—96 %—es la métrica más estable; el tiempo escala aún más cuando la tabla no cabe en caché.)

## 5. Conclusión general

| Fase                 | Técnica destacada                         | Mejor resultado                 |
|----------------------|-------------------------------------------|---------------------------------|
| 1 — Reescritura      | Anti-join <code>NOT EXISTS</code> (OPT-3) | -99,6 % tiempo (×273)           |
| 2 — Índices          | B-tree simple (IDX-1)                     | -99,7 % tiempo (×392)           |
| 3 — Particionamiento | Poda <code>RANGE</code> mensual           | -71,0 % tiempo, -96,1 % bloques |

Las tres palancas son **complementarias**: la reescritura corrige la complejidad algorítmica del SQL, los índices arreglan el camino de acceso de las consultas OLTP, y el particionamiento acota el volumen barrido y el mantenimiento de la tabla de hechos. Todas las métricas son reproducibles (`docker compose up + generate_data.py --seed 42`

- los scripts de `sql/`; las salidas crudas de `EXPLAIN` están en `results/`.

## 6. Información completa

El detalle íntegro —scripts SQL, generador de datos, `docker-compose`, capturas crudas de `EXPLAIN (ANALYZE, BUFFERS)` e instrucciones de reproducción— está en el repositorio:

<https://github.com/jorge-figueroa-eng/Ecommify-Database-Design> (carpeta `actividad4/`)

| Recurso                                                          | Ruta                                       |
|------------------------------------------------------------------|--------------------------------------------|
| Scripts SQL numerados (esquema, consultas, índices, particiones) | <code>actividad4/sql/</code>               |
| Generador de datos sintéticos                                    | <code>actividad4/generate_data.py</code>   |
| Entorno Docker (PostgreSQL 16 + PostGIS)                         | <code>actividad4/docker-compose.yml</code> |
| Salidas crudas de <code>EXPLAIN</code>                           | <code>actividad4/results/</code>           |
| Guía de reproducción                                             | <code>actividad4/README.md</code>          |